



LISTA DE EXERCÍCIOS - SISTEMAS INTELIGENTES APLICADOS

Lista de Exercícios 6

Aprendizado de Máquina

DOCENTE: Thiago França Naves

DATA: 30/09/2026

ALUNO: Mayara Rodrigues Pereira e Leonardo José Reis Pinto

Esta lista é teórica. Nas questões 2 e 4 fizemos um experimento curto com a base de dígitos 8x8 do scikit-learn (1797 imagens, 64 pixels), para mostrar na prática o que a resposta explica. O script está no Apêndice A.

Questão 1

Qual a diferença entre um classificador e um regressor? Exemplifique sua resposta para o problema de reconhecimento de dígitos.

Resposta. Os dois aprendem uma função a partir de exemplos rotulados. A diferença está na saída. O classificador devolve uma categoria, um valor de um conjunto finito sem noção de distância entre eles, e é avaliado por acurácia, precisão, recall e matriz de confusão. O regressor devolve um número contínuo, em que errar por pouco é melhor que errar por muito, e é avaliado por erros como o MSE ou o MAE.

No reconhecimento de dígitos, o problema natural é de classificação: a imagem entra e sai uma de dez classes, de 0 a 9. Confundir um 1 com um 7 é tão errado quanto confundir com um 2, porque os dígitos são símbolos e não quantidades. Um regressor que devolvesse 6,4 para a imagem de um 7 não faria sentido, e arredondar esse valor criaria erros esquisitos, como trocar 3 por 4 só porque os números são vizinhos. Dentro do mesmo problema há espaço para regressão em tarefas auxiliares, como prever a inclinação do dígito em graus ou as coordenadas da caixa que envolve cada dígito num formulário.

Questão 2

O que é a maldição da dimensionalidade, quando ela aparece e como se resolve? Exemplifique. Qual a relação entre algoritmos do tipo PCA (principal component analysis) com esse problema?

Resposta. É o conjunto de problemas que aparecem quando o número de atributos cresce. O volume do espaço cresce exponencialmente com a dimensão, então a mesma quantidade de exemplos fica cada vez mais espalhada. Para manter a mesma densidade de pontos, o número de exemplos precisaria crescer exponencialmente. Outro efeito é que as distâncias perdem o contraste: em muitas dimensões, todos os pontos ficam mais ou menos à mesma distância uns dos outros, e métodos baseados em vizinhança, como KNN e K-Means, deixam de distinguir perto de longe.

Ela aparece quando há muitos atributos em relação ao número de exemplos: texto representado por palavras (milhares de colunas), imagens tratadas pixel a pixel, dados de expressão gênica com milhares de genes e poucas dezenas de pacientes. Os sintomas são overfitting, modelos que exigem muitos dados e custo alto de processamento.

No experimento, sorteamos 500 pontos uniformes e medimos a razão entre a distância do vizinho mais longe e a do mais perto de um ponto de consulta. Em duas dimensões o mais longe está 83 vezes mais distante. Em 100 dimensões, apenas 1,43 vez, e em 1000, 1,10 vez.

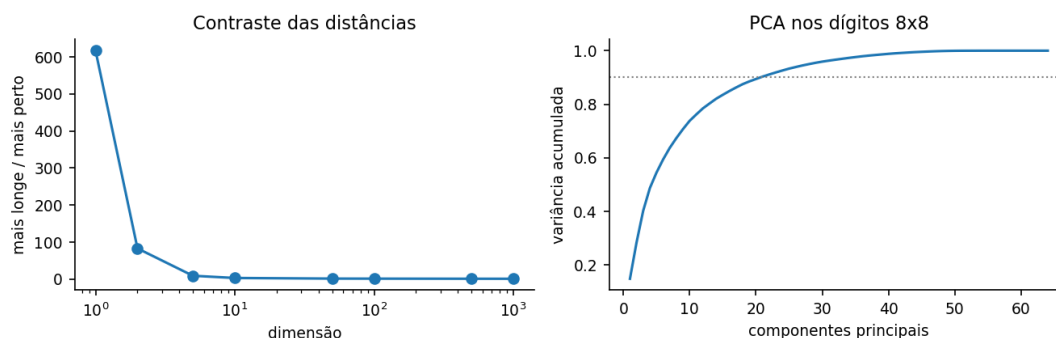


Figura 1. Esquerda: contraste entre as distâncias em função da dimensão. Direita: variância acumulada do PCA nos dígitos.

As soluções são reduzir a dimensão ou usar mais dados. Dá para selecionar atributos (ficar só com os relevantes), extrair atributos novos que resumam os originais, regularizar o modelo ou usar conhecimento do problema para montar atributos melhores.

O PCA é a forma mais comum de extração. Ele encontra as direções em que os dados variam mais (componentes principais, que são os autovetores da matriz de covariância) e projeta os dados nas primeiras delas. Muitos atributos originais são correlacionados, e o PCA junta essa informação repetida em poucos eixos independentes. Nos dígitos, 21 dos 64 componentes já guardam 90% da variância. Um KNN com 3 vizinhos teve 96,66% de acerto com os 64 pixels e 96,05% com só 20 componentes, três vezes menos dimensões para quase o mesmo resultado. Com 2 componentes cai para 58,8%, então reduzir demais também perde informação. A limitação do PCA é que ele é linear e não olha a classe: uma direção de pouca variância pode ser a que separa as classes.

Questão 3

Como um sistema de aprendizado de máquina pode ser implantado em um agente inteligente para que ele consiga tratar alguns sensores que estão falhando?

Resposta. O agente precisa de duas coisas: perceber que um sensor falhou e continuar funcionando sem ele. O aprendizado de máquina ajuda nas duas.

Para **detectar a falha**, o agente aprende como é o funcionamento normal dos sensores, com um modelo de detecção de anomalias (por exemplo, K-Means ou uma distribuição gaussiana ajustada às leituras normais). Os sensores de um robô ou de uma máquina são correlacionados: a velocidade das rodas combina com a posição do GPS, a temperatura de um ponto acompanha a do ponto vizinho. Um modelo de regressão treinado para prever cada sensor a partir dos outros denuncia a falha quando a leitura real se afasta muito da prevista, ou quando o valor fica travado, some ou dá saltos impossíveis.

Para **contornar a falha**, o mesmo regressor serve para estimar o valor do sensor com defeito a partir dos que funcionam (imputação), e a decisão do agente usa a estimativa no lugar da leitura.

Outra opção é treinar o modelo de decisão já prevendo falhas, desligando sensores aleatoriamente durante o treino, para ele aprender a decidir com um subconjunto qualquer das entradas. Isso junto com redundância de sensores e fusão das leituras (um filtro de Kalman, por exemplo) deixa o agente degradar aos poucos em vez de parar de uma vez. Como o desgaste muda o comportamento dos sensores com o tempo, o ideal é que o modelo continue aprendendo com os dados novos durante a operação.

Questão 4

Explique como funciona a validação cruzada e qual sua utilidade. Essa técnica resolve o problema do super treinamento (over training)? Garante a generalização do classificador? Justifique.

Resposta. Na validação cruzada com k dobras, a base é dividida em k partes de tamanho parecido, normalmente mantendo a proporção das classes (estratificada). O classificador é treinado k vezes, cada vez com $k - 1$ partes, e testado na parte que ficou de fora. A estimativa final é a média dos k testes. Todo exemplo é usado uma vez para teste e $k - 1$ vezes para treino. Com k igual ao número de exemplos, é o “deixa um de fora”.

A utilidade é estimar o desempenho em dados novos sem gastar uma parte fixa da base só para teste, o que importa quando há poucos dados, e com menos dependência da sorte de uma divisão única. Também serve para comparar algoritmos e escolher hiperparâmetros.

Ela **não resolve** o overfitting, ela **mostra** o overfitting. O modelo continua sendo treinado do mesmo jeito. No experimento, uma árvore de decisão sem limite de profundidade acerta 100% do próprio treino, mas só 82,4% na validação cruzada. A curva mostra que a partir da profundidade 10 o acerto no treino continua subindo e o da validação para. Com essa informação é possível escolher a profundidade, a poda ou o momento de parar o treino, e aí sim evitar o super treinamento.

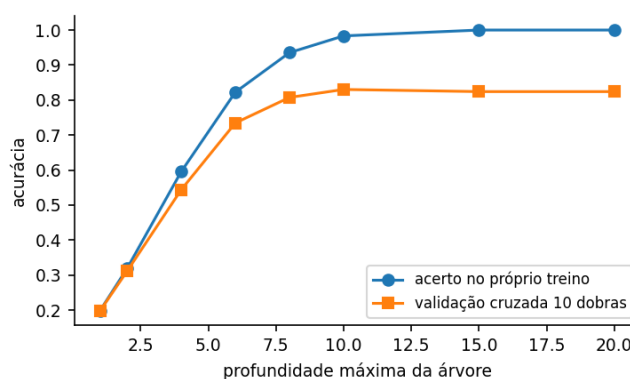


Figura 2. Acurácia no treino e na validação cruzada conforme a profundidade da árvore cresce.

Ela também **não garante** a generalização. A estimativa só vale para dados que venham da mesma distribuição da base. Se a base for enviesada, pequena demais, tiver exemplos repetidos entre dobras ou se o mundo mudar depois (spam novo, sensor diferente), o resultado da validação cruzada será otimista. E se a validação cruzada for usada muitas vezes para escolher hiperparâmetros, ela mesma pode sofrer overfitting, e por isso o ideal é guardar um conjunto de teste que só é usado no final.

Questão 5

O problema de detecção de spams tem sido tratado de diversas formas nos últimos anos. Discuta um classificador para detecção de spam considerando dois cenários distintos: a. Muitos dados de ambas as classes. b. Poucos dados.

Resposta. a) Muitos dados. Com muitos e-mails rotulados de spam e de não spam, o problema é uma classificação supervisionada comum. Cada e-mail vira um vetor de atributos (frequência de palavras com TF-IDF, remetente, links, horário, cabeçalhos). Em cima disso funcionam bem uma regressão logística, uma SVM linear ou, com volume grande, uma rede neural que aprende a representação do texto direto. Alguns cuidados: a base deve ter a proporção real de spam, os erros não custam o mesmo (mandar um e-mail importante para a lixeira é pior que deixar passar um spam, então o limiar de decisão deve ser ajustado para precisão alta na classe spam) e o modelo precisa ser retreinado sempre, porque os spammers mudam o texto para escapar do filtro.

b) Poucos dados. Modelos com muitos parâmetros sofreriam overfitting. O Naive Bayes multinomial é a escolha clássica: ele só estima frequências de palavras por classe, funciona com poucas centenas de exemplos e foi um dos primeiros filtros de spam usados na prática. Também dá para aproveitar dados sem rótulo com aprendizado semissupervisionado, usar representações de texto pré-treinadas em outras bases (transferência de aprendizado), pedir ao usuário o rótulo só dos e-mails em que o modelo está em dúvida (aprendizado ativo) e juntar o modelo com regras simples, como listas de bloqueio. Se só houver exemplos de e-mails legítimos, um classificador de uma classe pode tratar o spam como anomalia.

Questão 6

O que é aprendizado não supervisionado e como ele se beneficia de técnicas de redução da dimensionalidade no contexto específico de técnicas de agrupamento?

Resposta. É o aprendizado a partir de dados sem rótulo. Não há uma resposta certa para imitar, e o objetivo é encontrar estrutura nos próprios dados: grupos de exemplos parecidos (agrupamento, como o K-Means), regras de associação ou uma representação mais compacta.

O agrupamento depende de uma medida de distância, e é aí que a redução de dimensionalidade ajuda:

- em muitas dimensões as distâncias ficam parecidas (questão 2), e os grupos deixam de ser nítidos. Projetar nas direções principais devolve contraste;
- atributos correlacionados contam a mesma informação várias vezes e distorcem a distância. O PCA gera eixos sem correlação;
- atributos com ruído são descartados junto com os componentes de pouca variância;
- o custo do K-Means cresce com o número de atributos, e com menos dimensões cada iteração fica mais rápida;
- com 2 ou 3 componentes é possível ver os grupos num gráfico, o que ajuda a escolher o número de grupos e explicar o resultado.

O cuidado é o mesmo da questão 2: a redução não usa rótulo e pode descartar uma direção pequena que separava dois grupos, então é bom comparar o agrupamento com e sem a redução.

Apêndice A. Código

```
pip install numpy matplotlib scikit-learn
python solucao.py

# Lista 6 - Aprendizado de Máquina
# Sistemas Inteligentes Aplicados - UTFPR
# Experimentos curtos para as questões 2 e 4
# Uso: python solucao.py

import os
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

PASTA = os.path.dirname(os.path.abspath(__file__))
SAIDA = os.path.join(PASTA, "saidas")
os.makedirs(SAIDA, exist_ok=True)
plt.rcParams.update({"font.size": 9, "axes.spines.top": False,
                    "axes.spines.right": False})
rng = np.random.default_rng(0)

# questao 2: distâncias perdem o contraste quando a dimensão cresce
print("[2] razão entre o vizinho mais longe e o mais perto")
dims = [1, 2, 5, 10, 50, 100, 500, 1000]
razoes = []
for d in dims:
    pontos = rng.random((500, d))
    consulta = rng.random(d)
    dist = np.linalg.norm(pontos - consulta, axis=1)
    razoes.append(dist.max() / dist.min())
    print(f" d={d:4d}: mais longe / mais perto = {razoes[-1]:.2f}")

# PCA nos dígitos 8x8: quanto da variância fica com poucos componentes
X, y = load_digits(return_X_y=True)
pca = PCA().fit(X)
acum = np.cumsum(pca.explained_variance_ratio_)
for p in [0.80, 0.90, 0.95]:
    print(f" {p:.0%} da variância com {np.searchsorted(acum, p) + 1}"
          " de 64 componentes")

knn = KNeighborsClassifier(3)
base = cross_val_score(knn, X, y, cv=5).mean()
print(f" KNN k=3 com os 64 pixels: {base:.4f}")
for n in [2, 5, 10, 20, 30]:
    Xr = PCA(n).fit_transform(X)
    print(f" KNN k=3 com {n:2d} componentes: "
          f"{cross_val_score(knn, Xr, y, cv=5).mean():.4f}")

fig, ax = plt.subplots(1, 2, figsize=(9, 3))
ax[0].semilogx(dims, razoes, "o-")
ax[0].set_xlabel("dimensão")
ax[0].set_ylabel("mais longe / mais perto")
ax[0].set_title("Contraste das distâncias")
ax[1].plot(range(1, 65), acum)
ax[1].axhline(0.9, color="gray", ls=":", lw=1)
ax[1].set_xlabel("componentes principais")
ax[1].set_ylabel("variância acumulada")
ax[1].set_title("PCA nos dígitos 8x8")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q2_dimensionalidade.png"), dpi=150)
plt.close(fig)

# questao 4: validação cruzada mostra o overfitting que o treino esconde
print("\n[4] árvore de decisão nos dígitos")
```

```

prof = [1, 2, 4, 6, 8, 10, 15, 20]
treino = []
cv = []
for p in prof:
    arv = DecisionTreeClassifier(max_depth=p, random_state=0)
    treino.append(arv.fit(X, y).score(X, y))
    cv.append(cross_val_score(arv, X, y, cv=10).mean())
    print(f" profundidade {p:2d}: treino={treino[-1]:.4f} cv10={cv[-1]:.4f}")

fig, ax = plt.subplots(figsize=(5, 3))
ax.plot(prof, treino, "o-", label="acerto no próprio treino")
ax.plot(prof, cv, "s-", label="validação cruzada 10 dobras")
ax.set_xlabel("profundidade máxima da árvore")
ax.set_ylabel("acurácia")
ax.legend(fontsize=8)
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "q4_validacao.png"), dpi=150)
plt.close(fig)

```